Rockchip Andoid15 SecurityBoot AVB Developer Guide

ID: RK-KF-YF-E19

Release Version: V1.1.0

Release Date: 2025-04-02

Security Level: □Secret □Internal □Public ■Public

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document introduces the boot methods for SecurityBoot and AVB features based on Android15.

Intended Audience

This document (this guide) is mainly intended for:

Technical Support Engineer

Software Development Engineer

Revision History

Version	Author	Date	Change Description
V1.0.0	Wu Liangqing	2024-11-11	Initial version release
V1.1.0	Wu Liangqing	2025-04-02	Added oem unlock process

Contents

1. Code Environment

This document is applicable to Android 15 and later versions, including the chip sets:

- RK3588 Series
- RK3576 Series
- RK356x Series
- RK3326 Series
- PX30 Series
- RK3399 series (applicable only to the AVB portion, the SecurityBoot portion refers to the document《Rockchip_RK3399_User_Guide_SecurityBoot_And_AVB_CN.md》)

2. Secure Boot SecurityBoot Operation Steps

2.1 Compilation Environment Confirmation

Confirm whether the version of the fdtput on the compilation server is 1.4.5.

```
#fdtput --version
#Version: DTC 1.4.5
```

If the fdtput version is less than 1.4.5, please execute the following command to upgrade:

```
sudo apt-get install device-tree-compiler
```

2.2 Enter the u-boot directory

```
cd u-boot
```

All the following operation steps are executed in the u-boot directory.

2.3 Code Modification

Enter the u-boot directory, open the configs/rk3588_defconfig for the corresponding platform, and select the following configurations:

```
// Edit the configs/rk3588_defconfig file, both RK3588 and RK3588s are modified through this file
vim configs/rk3588_defconfig
// Mandatory.
CONFIG_FIT_SIGNATURE=y
CONFIG_SPL_FIT_SIGNATURE=y
CONFIG_AVB_VBMETA_PUBLIC_KEY_VALIDATE=y

// Optional.
CONFIG_FIT_ROLLBACK_PROTECT=y       // boot.img anti-rollback
CONFIG_SPL_FIT_ROLLBACK_PROTECT=y   // uboot.img anti-rollback
```

2.4 Generating Keys

In the u-boot directory, perform the following operations to generate the keys:

```
mkdir -p keys
../rkbin/tools/rk_sign_tool kk --bits 2048 --out .
cp privateKey.pem keys/dev.key && cp publicKey.pem keys/dev.pubkey
openssl req -batch -new -x509 -key keys/dev.key -out keys/dev.crt
```

Note: This step needs to be executed only once. Afterwards, ensure the proper preservation of these keys.

2.5 Compilation Signature

Take RK3588 as an example:

```
./make.sh rk3588 --spl-new --rollback-index-uboot 1 --burn-key-hash
```

Note:

--spl-new //Repackage the signed spl

--rollback-index-uboot // Set the version number. When the config in step 2 enables anti-rollback, this compiling option must be added, otherwise it is unnecessary.

--burn-key-hash // adding this compiling option will burn the chip fuse during startup after flashing the firmware.

If compilation encounters:

```
Can't load XXXXXX//.rnd into RNG
```

Execute:

```
touch ~/.rnd
```

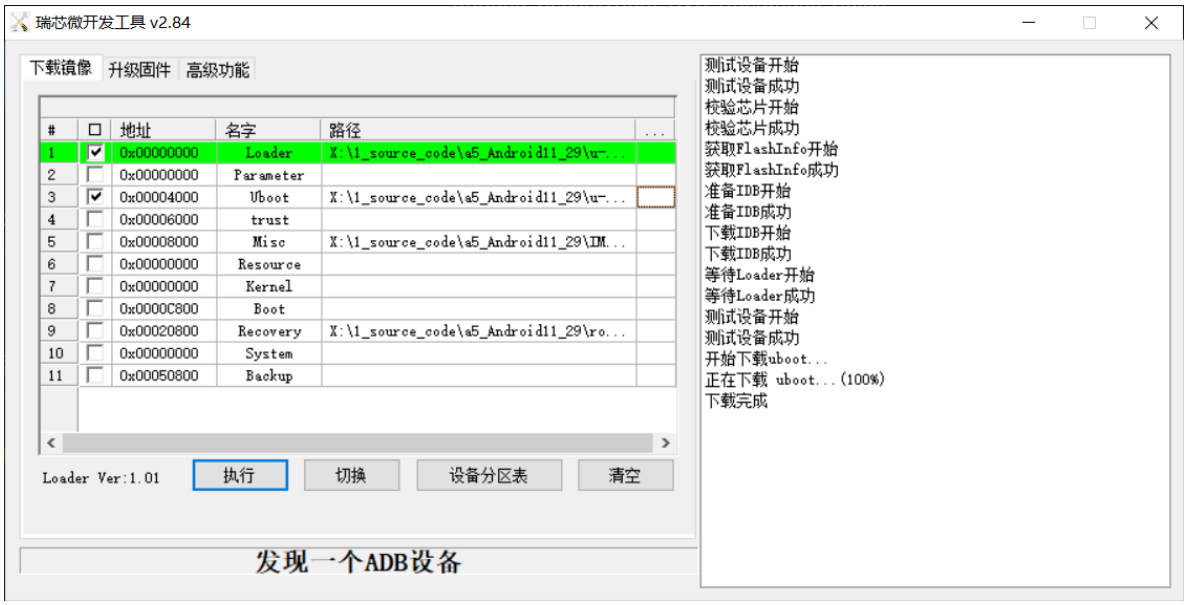
2.6 Compiling the Complete Firmware

Compile other firmware using the standard firmware compilation method (uboot and loader have already been compiled, do not need to be recompiled), such as using

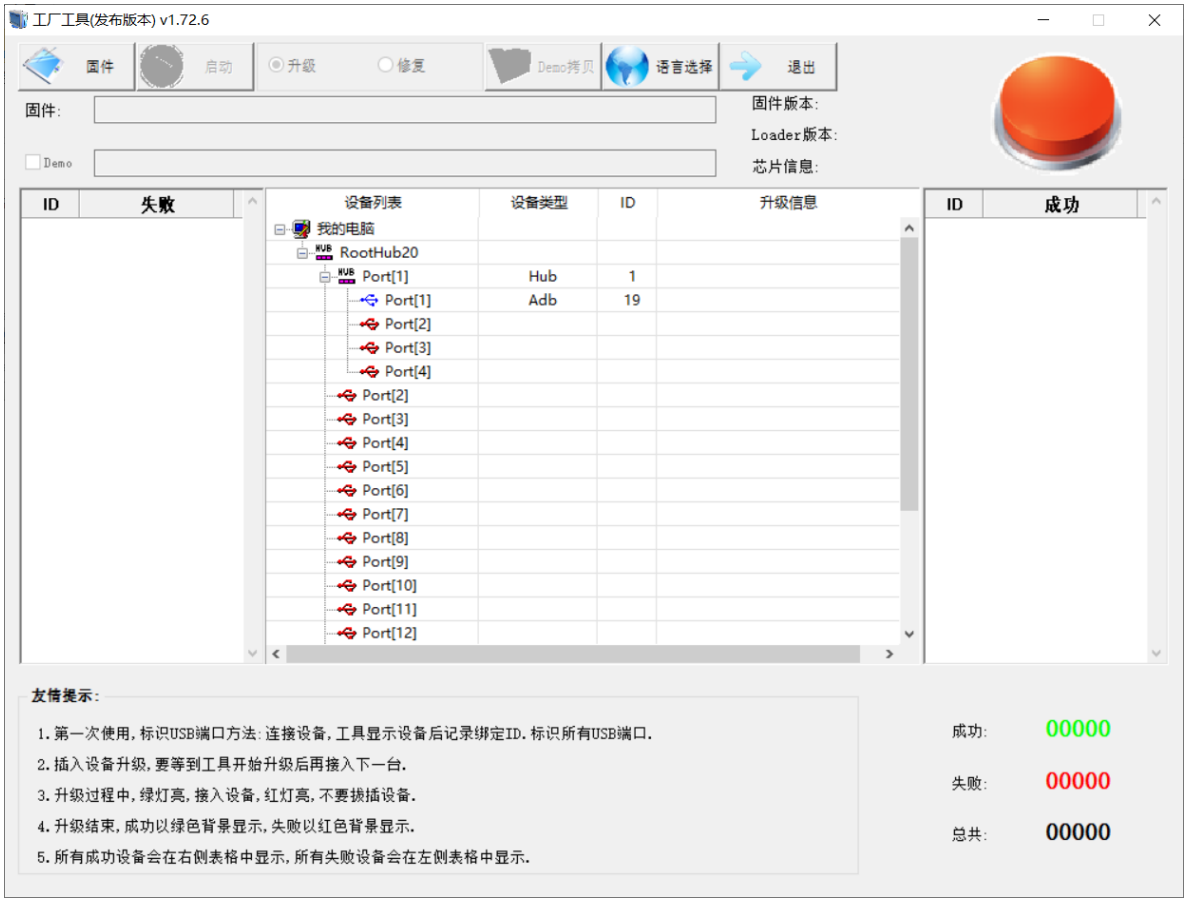
```
source build/envsetup.sh
lunch rk3588_s-userdebug
./build.sh -ACKup
```

2.7 Firmware Flashing

During the development and validation phase, uboot and loader can be flashed individually using AndroidTool.



In the mass production phase, use the mass production tool to flash the update.img generated by compiling step 5.



2.8 Determine if Fusing is Successful

2.8.1 Check via Boot Serial Port Logs

```
### Verified-boot: 0   // Firmware is signed but chip fuses are not burned (1
indicates fused), no hash-based validation performed, i.e., compilation was done
without the `--burn-key-hash` option
sha256,rsa2048:dev+
rollback index: 1 >= 0(min), OK   // rollback version, i.e., add --rollback-
index-uboot when compiling
// Below are the integrity checks for U-Boot.
## Checking atf-1 0x00040000 ... sha256+ OK
## Checking uboot 0x00a00000 ... sha256+ OK
## Checking fdt 0x00b2a018 ... sha256+ OK
## Checking atf-2 0xfdcc9000 ... sha256+ OK
## Checking atf-3 0xfdcd0000 ... sha256+ OK
## Checking optee 0x00200000 ... sha256+ OK
```

2.8.2 Flash unsigned loader and uboot for verification

3. Android Verified Boot (AVB) Operation Steps

3.1 Compile the avbtool Tool

```
mmma external/avb/ -j16
```

Generated after compilation completes:

```
out/host/linux-x86/bin/avbtool
```

3.2 Generate cert_permanent_attributes.bin

- **Modify Product ID**

```
cd external/avb/test
```

```
diff --git a/test/avb_cert_generate_test_data b/test/avb_cer_generate_test_data
index 1b8bb2b..2220688 100755
--- a/test/avb_cer_generate_test_data
+++ b/test/avb_cer_generate_test_data
@@ -48,7 +48,7 @@ AVBTOOL=$(dirname "$0")/../avbtool
 echo AVBTOOL = ${AVBTOOL}

 # Get a zero product ID.
-echo 00000000000000000000000000000000 | xxd -r -p - cert_product_id.bin
+echo 00000000000000000000000000000123 | xxd -r -p - cert_product_id.bin

 # Generate key pairs.
 if [ ! -f testkey_cer_prk.pem ]; then
```

```
cd -
```

Note: The Product ID must be 16 digits in length, and its numerical value can be defined by the user.

- **Generate cert_permanent_attributes.bin**

```
cd external/avb/test/data
```

```
../avb_cert_generate_test_data
```

```
cd -
```

After performing the above operations, the following will be generated in the directory external/avb/test/data:

- cert_permanent_attributes.bin
- cert_metadata.bin
- testkey_cert_pik.pem
- testkey_cert_prk.pem
- testkey_cert_psk.pem
- testkey_cert_puk.pem

Note:

- The system has a PEM file by default. If regeneration is required, the system's default file must be deleted first, then execute the aforementioned steps to regenerate the PEM file. It is recommended that customers regenerate it themselves.
- This step only needs to be executed once per product. Please properly preserve the files generated above, which will be used in the subsequent steps.

3.3 Code Modifications

```
cd device/rockchip/rk3588
```

```
diff --git a/rk3588_s/BoardConfig.mk b/rk3588_s/BoardConfig.mk
index 24b415f..80fa60f 100644
--- a/rk3588_s/BoardConfig.mk
+++ b/rk3588_s/BoardConfig.mk
```

```
@@ -37,3 +37,7 @@ ifeq ($(strip $(BOARD_USES_AB_IMAGE)), true)
   include device/rockchip/common/BoardConfig_AB.mk
   TARGET_RECOVERY_FSTAB := device/rockchip/rk3588/rk3588_s/recovery.fstab_AB
 endif
+
+BOARD_AVB_ENABLE := true    // Enable AVB functionality
+BOARD_AVB_ALGORITHM := SHA256_RSA4096  // Configure encryption algorithm
+BOARD_AVB_KEY_PATH := external/avb/test/data/testkey_cert_psk.pem  // secret key storage path
+BOARD_AVB_METADATA_BIN_PATH := external/avb/test/data/cert_metadata.bin // specifies the metadata file
+#BOARD_AVB_ROLLBACK_INDEX := 5   // Configure rollback prevention, disabled by default. Enable based on requirements and requires modification in U-Boot
```

```
cd -
cd u-boot
```

```
diff --git a/configs/rk3588_defconfig b/configs/rk3588_defconfig
index 3017921487..84197eea1e 100644
--- a/configs/rk3588_defconfig
+++ b/configs/rk3588_defconfig
@@ -214,5 +214,9 @@ CONFIG_AVB_LIBAVB_AB=y
 CONFIG_AVB_LIBAVB_cert=y
 CONFIG_AVB_LIBAVB_USER=y
 CONFIG_RK_AVB_LIBAVB_USER=y
+CONFIG_RK_AVB_LIBAVB_ENABLE_ATH_UNLOCK=y
+CONFIG_AVB_VBMETA_PUBLIC_KEY_VALIDATE=y
//Optional Configuration
+CONFIG_ANDROID_AVB_ROLLBACK_INDEX=y // Rollback prevention feature. Configure this option only if required, and ensure it is configured together with BOARD_AVB_ROLLBACK_INDEX under the device.
```

```
cd -
```

3.4 AVB Key Flashing

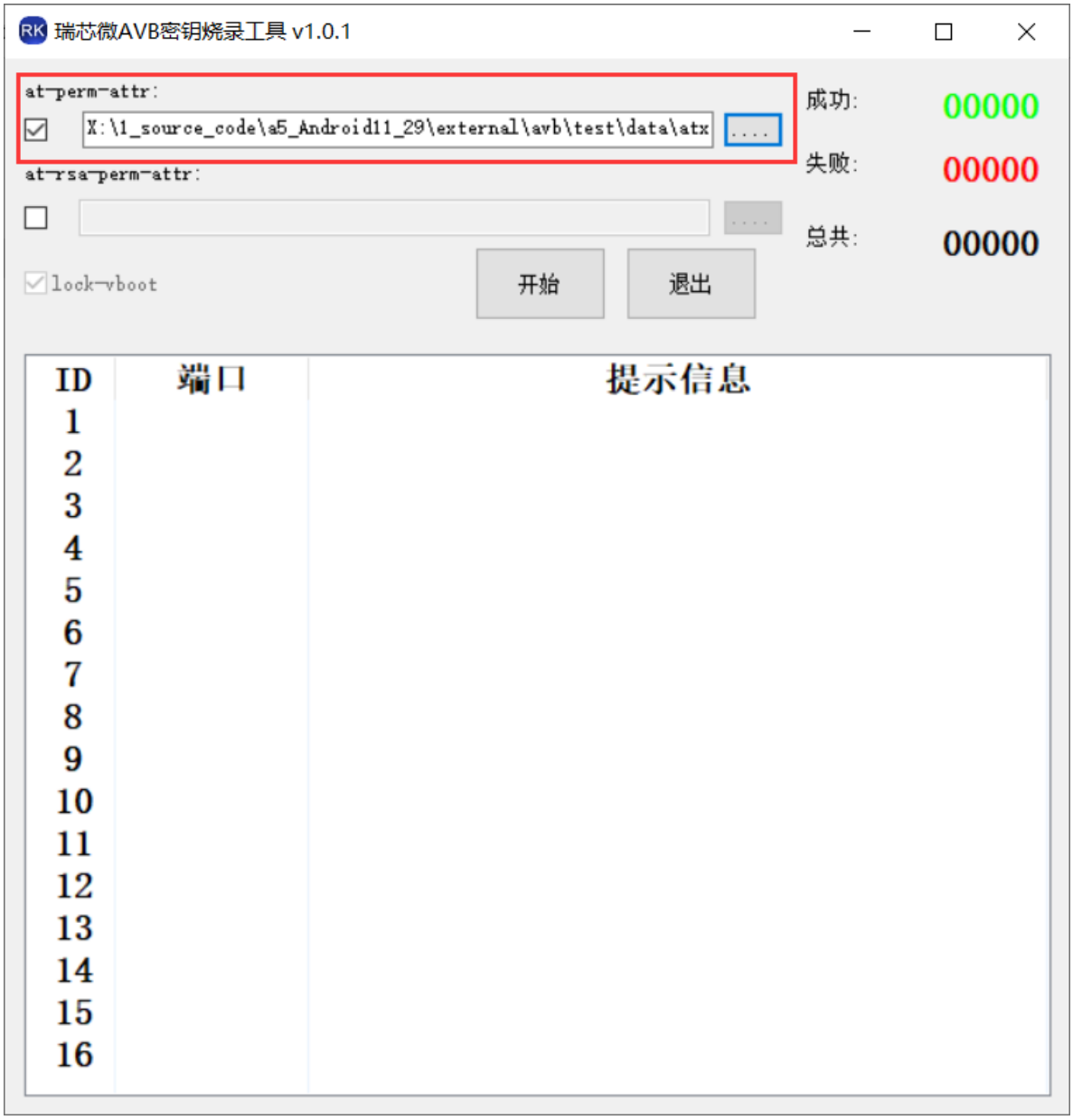
3.4.1 Method 1: Flashing via AVB Key Flashing Tool

Flashing tool: AvbKeyWriter (RKTools/windows/AvbKeyWriter-v1.0.1.7z)

Flashing source files: The external/avb/test/data produced in Step 1 generates cert_permanent_attributes.bin

Flashing Methods:

- Check at-perm-attr
- Import the external/avb/test/data produced in Step 3 to generate cert_permanent_attributes.bin
- The device to be flashed enters loader mode
- Click "Power-On Button to Start Flashing"



3.4.2 Method Two: AVB key is integrated into the U-Boot code and flashed into the device along with the firmware, no need to use an AVB key tool to flash the AVB key

This method embeds the key directly within the uboot.img firmware without flashing it into a secure partition. When the boot.img is replaced, the key becomes invalid. Therefore, it must be used together with security boot mechanisms to ensure the firmware in the uboot partition not to be altered or replaced.

Note: This method does not support unlocking functionality with strict AVB verification, so please use Method One for unlocking.

3.4.2.1 This method is not supported by default in the SDK and requires to apply the patch in the uboot

Patch path: RKDocs/common/security/patch/u-boot/0001-avb-add-embedded-key.patch

```
cd u-boot
```

```
git am RKDocs/common/security/patch/u-boot/0001-avb-add-embedded-key.patch
```

```
cd -
```

3.4.2.2 Extract Public Key

```
avbtool extract_public_key --key external/avb/test/data/testkey_cert_psk.pem --
output avb_root_pub.bin
xxd -i avb_root_pub.bin > external/avb/test/data/avb_root_pub.h
```

3.4.2.3 Replace the Public Key

Replace the avb_root_pub array in u-boot/lib/avb/libavb_user/avb_ops_user.c with the extracted public key (external/avb/test/data/avb_root_pub.h).

```
cd u-boot
```

```
vim lib/avb/libavb_user/avb_ops_user.c
```

```
/**
* Internal builds use testkey_rsa4096.pem
* OEM should replace this Array with public key used to sign vbmeta.img
* *
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 \
* -outform PEM -out avb_rsa4096.pem
* avbtool extract_public_key --key avb_rsa4096.pem --output avb_root_pub.bin
* xxd -i avb_root_pub.bin > avb_root_pub.h
*/
static const char avb_root_pub [] = {
0x00, 0x00, 0x10, 0x00, 0x55, 0xd9, 0x04, 0xad, 0xd8, 0x04, 0xaf, 0xe3,
0xd3, 0x84, 0x6c, 0x7e, 0x0d, 0x89, 0x3d, 0xc2, 0x8c, 0xd3, 0x12, 0x55,
0xe9, 0x62, 0xc9, 0xf1, 0x0f, 0x5e, 0xcc, 0x16, 0x72, 0xab, 0x44, 0x7c,
0x2c, 0x65, 0x4a, 0x94, 0xb5, 0x16, 0x2b, 0x00, 0xbb, 0x06, 0xef, 0x13,
0x07, 0x53, 0x4c, 0xf9, 0x64, 0xb9, 0x28, 0x7a, 0x1b, 0x84, 0x98, 0x88,
0xd8, 0x67, 0xa4, 0x23, 0xf9, 0xa7, 0x4b, 0xdc, 0x4a, 0x0f, 0xf7, 0x3a,
0x18, 0xae, 0x54, 0xa8, 0x15, 0xfe, 0xb0, 0xad, 0xac, 0x35, 0xda, 0x3b,
0xad, 0x27, 0xbc, 0xaf, 0xe8, 0xd3, 0x2f, 0x37, 0x34, 0xd6, 0x51, 0x2b,
… … …
```

```
cd -
```

3.5 Firmware Compilation

After completing the above steps, a full firmware compilation can be performed. Using the RK3566_r product as an example for compilation:

- **Secure Boot-enabled solution**

The U-Boot with Secure Boot must be compiled separately first. Refer to the operational steps of the Secure Boot mentioned above.

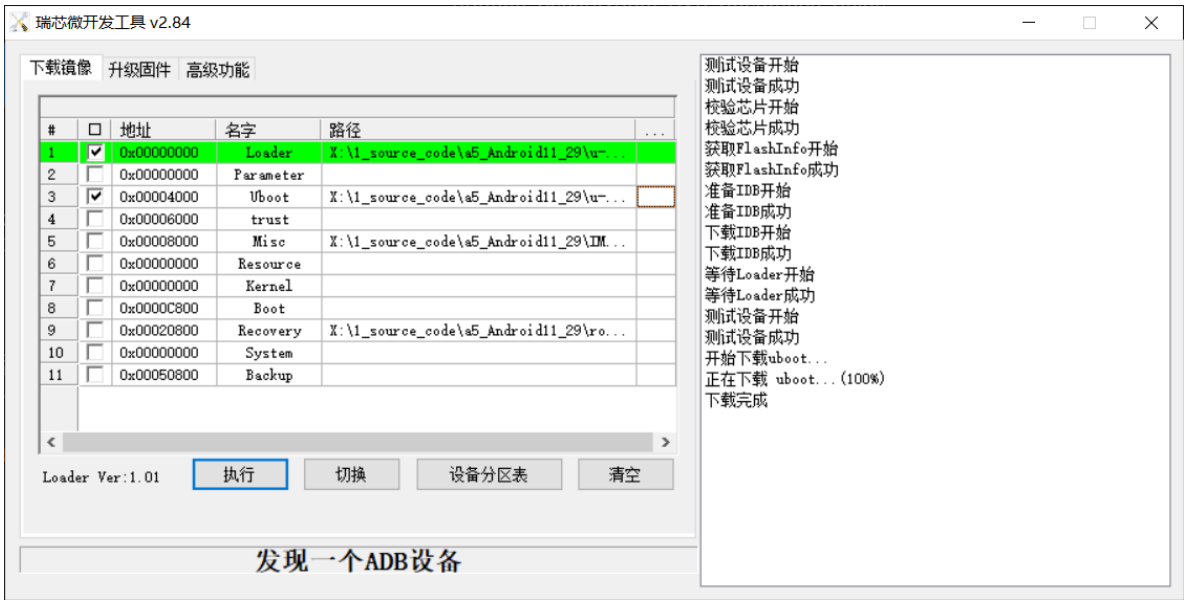
```
source build/envsetup.sh
lunch rk3588_s-userdebug
./build.sh -ACKup
```

- **Solutions without Secure Boot**

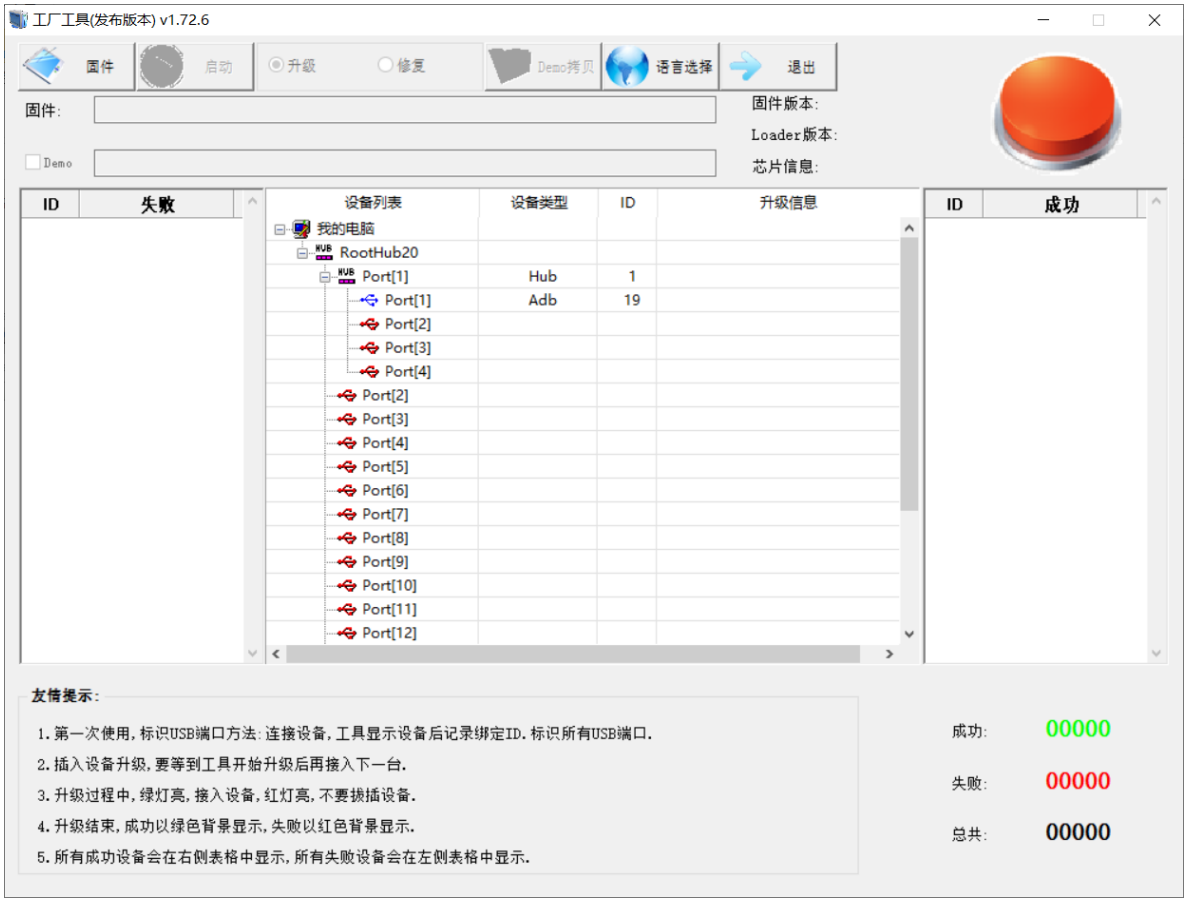
```
source build/envsetup.sh
lunch rk3588_s-userdebug
./build.sh -ACKUup
```

3.6 Firmware Flashing

During the development phase, use the AndroidTool tool to flash the compiled full firmware.



During the mass production phase, use the mass production tool to flash the update.img file compiled in Step 4.



3.7 Startup Validation

- **uboot boot log confirmation**

After completing the above steps, during system boot, the following printouts will be displayed in the serial port logs during the u-boot phase:

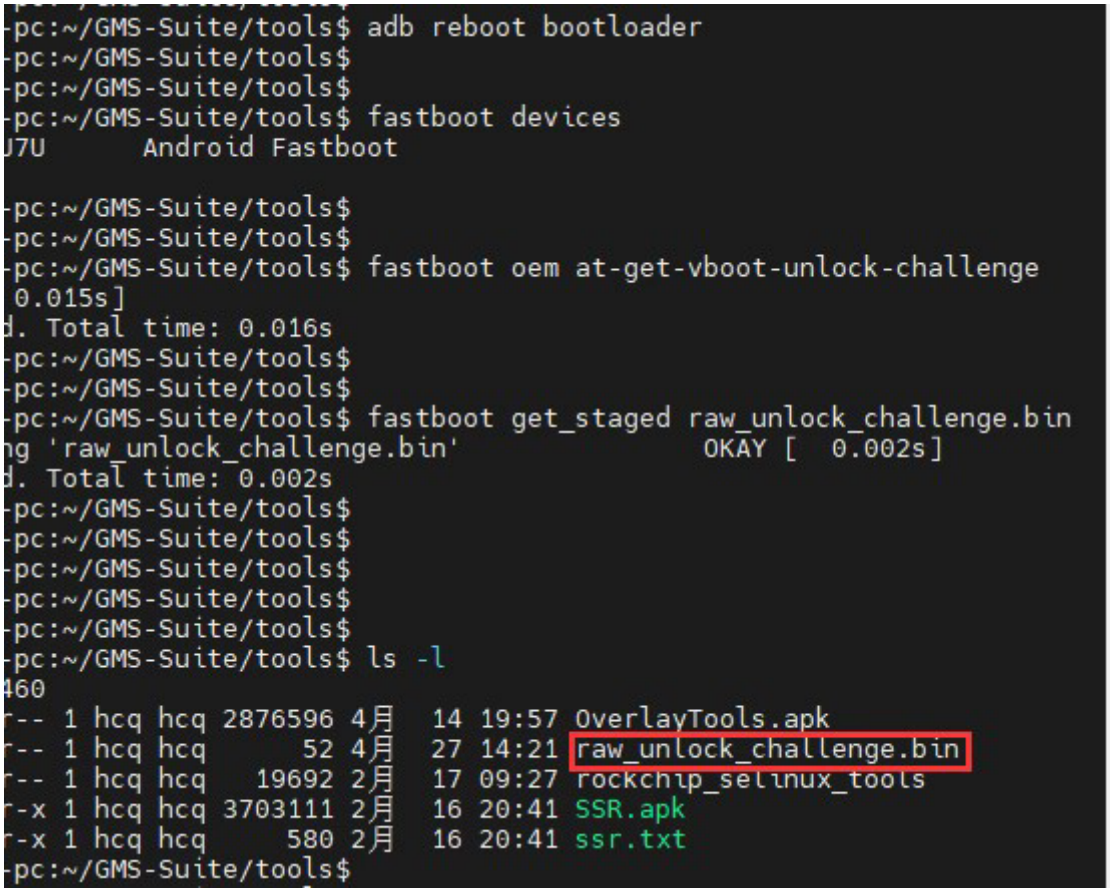
```
Vboot=0, AVB images, AVB verify
read_is_device_unlocked() ops returned that device is LOCKED
ANDROID: Hash OK
```

- **Flashing non-AVB firmware or other firmware will result in the device failing to boot.**

3.8 Unlocking Steps with AVB Strict Validation

- Enter fastboot mode on the device, and input on the computer:

```
adb reboot bootloader
fastboot oem at-get-vboot-unlock-challenge
fastboot get_staged raw_unlock_challenge.bin
```



Obtain the data containing the version, Product ID, and a 16-byte random number. Extract the random number to generate raw_unlock_challenge.bin.

- Use avbtool to generate unlock_credential.bin

Based on raw_unlock_challenge.bin and cert_product_id.bin, where cert_product_id.bin was generated when generating the cert_permanent_attributes.bin earlier.

```
python avb-challenge-verify.py raw_unlock_challenge.bin cert_product_id.bin
```

```
python avbtool make_unlock_credential --output=unlock_credential.bin --
intermediate_key_certificate=pik_certificate.bin --
unlock_key_certificate=puk_certificate.bin --challenge=unlock_challenge.bin --
unlock_key=testkey_puk.pem
```

The implementation code for avb-challenge-verify.py is as follows. Please copy the code and save it as avb-challenge-verify.py.

```
```

```
#/user/bin/env python
"This is a test module for getting unlock_challenge.bin"
import sys
import os
from hashlib import sha256

def challenge_verify():
    if (len(sys.argv) != 3) :
        print "Usage: rkpublickey.py [challenge_file] [product_id_file]"
        return
    if ((sys.argv[1] == "-h") or (sys.argv[1] == "--h")):
        print "Usage: rkpublickey.py [challenge_file] [product_id_file]"
        return
    try:
        challenge_file = open(sys.argv[1], 'rb')
        product_id_file = open(sys.argv[2], 'rb')
        challenge_random_file = open('unlock_challenge.bin', 'wb')
        challenge_data = challenge_file.read(52)
        product_id_data = product_id_file.read(16)
        product_id_hash = sha256(product_id_data).digest()
        print("The challege version is %d" %ord(challenge_data[0]))
        if (product_id_hash != challenge_data[4:36]) :
            print("Product id verify error!")
            return
        challenge_random_file.write(challenge_data[36:52])
        print("Success!")

    finally:
        if challenge_file:
            challenge_file.close()
        if product_id_file:
            product_id_file.close()
        if challenge_random_file:
            challenge_random_file.close()
if __name__ == '__main__':
    challenge_verify()
```

- Flash unlock_credential.bin, and input on the computer:

```
fastboot stage unlock_credential.bin
fastboot oem at-unlock-vboot
```

Note: At this point, the device remains in the initial state of the first entry into fastboot mode. During this period, the device must not be powered off, shut down, or rebooted. This is because after completing Step 1, the device stores the generated random number. If power off, shutdown, or reboot occurs, it will result in the loss of the random number, causing subsequent validation of the challenge signature to fail due to mismatched random numbers.

If enabled:

```
CONFIG_MISC=y
CONFIG_ROCKCHIP_EFUSE=y
CONFIG_ROCKCHIP_OTP=y
```

The system will use the CPUID as the challenge number, since the CPUID is device-specific. The data persists even after system shutdown, and the generated `unlock_credential.bin` can be reused. This eliminates the need to regenerate `unlock_challenge.bin` and the steps to create `unlock_credential.bin`. The re-unlock process is simplified to:

```
fastboot oem at-get-vboot-unlock-challenge
fastboot stage unlock_credential.bin
fastboot oem at-unlock-vboot
```